

Documento de Prueba Unitaria — Lista Negra de Vehículos (xUnit)

Proyecto: ANPR-VISION (Backend)

Módulo: Business → `BlackListBusiness.Save`

Versión del documento: 1.0

Autor: Karol Natalia Osorio Poveda

Fecha de ejecución: 08/09/2025

Duración estimada: 10–15 min

Tipo de prueba: Unitaria

Herramientas: .NET SDK (9), xUnit, Moq, FluentAssertions, Test Explorer

Repositorio/Ubicación de pruebas: `tests/AnprVision.Business.Tests/BlackListBusiness_Tests.cs`

1. Resumen ejecutivo

Validar la lógica de negocio para **registro en Lista Negra** de vehículos. Se cubren tres escenarios clave:

- **Caso válido:** agregar un vehículo con **razón** y **fecha de restricción** → **se guarda** y devuelve DTO exitoso.
- **Caso inválido:** intentar agregar **sin razón** → **debe lanzar** `ArgumentException` (validación).
- **Regla de negocio:** un **mismo vehículo no puede estar dos veces** en la lista negra → al intentar duplicar, **debe fallar** con `InvalidOperationException`.

Resultado esperado global: El sistema **solo** guarda registros válidos, **rechaza** entradas sin razón y **bloquea** duplicados por `vehicleId` activo.

2. Reglas/Req. de negocio cubiertos

- La **razón** es **obligatoria** y su longitud máxima es 250 caracteres.
- Un **vehículo no puede registrarse dos veces** en la lista negra.
- Al guardar, se fija automáticamente `RestrictionDate` con `UtcNow` y `Asset = true`.

Fuera de alcance de esta prueba:

- Integridad referencial real (se asume un vehículo existente mediante `IVehicleBusiness.GetById`).
- Persistencia real en DB e índices únicos (validado vía integración, no en unit test).
- Borrado lógico/filtros por `IsDeleted` (si aplica).

3. Entorno y dependencias

- **Entorno:** Local Dev.
- **Dependencias moqueadas:**
 - `IBlackListData` → `ExistsAsync(...)` , `Save(...)` (persistence stub).
 - `IVehicleBusiness` → verificación de vehículo existente.
 - `IMapper` → mapeos `BlackListDto` ↔ `BlackList` (se usa configuración mínima en pruebas).

Artefacto bajo prueba (SUT): `BlackListBusiness`

Método: `Task<BlackListDto> Save(BlackListDto dto)`

Efectos esperados: Validaciones previas → guardado → DTO resultante con `Id` asignado.

4. Datos de entrada (mock)

- **Vehículos válidos:** `VehicleId` = 100 , 101 , 200 .
- **Razones de prueba:**
 - Válida: "Intento de fraude en acceso" , "Reportes previos de incidente" .
 - Inválida: "" (vacío o whitespace).

5. Casos de prueba

CT-01 — Caso válido: guarda y retorna DTO

Precondición: `VehicleId` = 100 existe; `ExistsAsync(...)` → `false` .

Pasos:

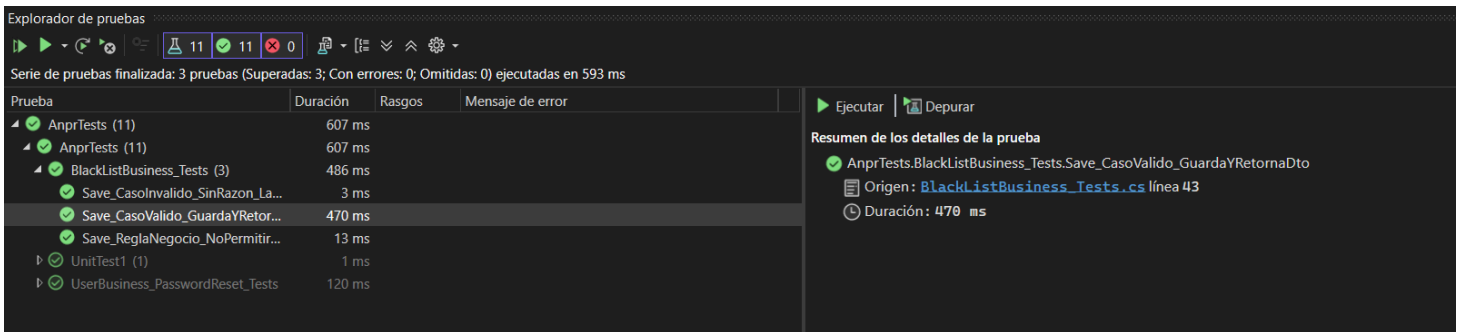
1. `Reason` = "Intento de fraude en acceso" .

2. Ejecutar `Save(dto)` .

3. **Assert:** DTO con `Id` asignado (55 simulado), `VehicleId` = 100 , `Reason` igual a la entrada, `RestrictionDate` != default .

Resultado esperado: ✓ Prueba exitosa.

Evidencia:



CT-02 — Caso inválido: sin razón → excepción de validación

Precondición: `VehicleId` = 101 existe.

Pasos:

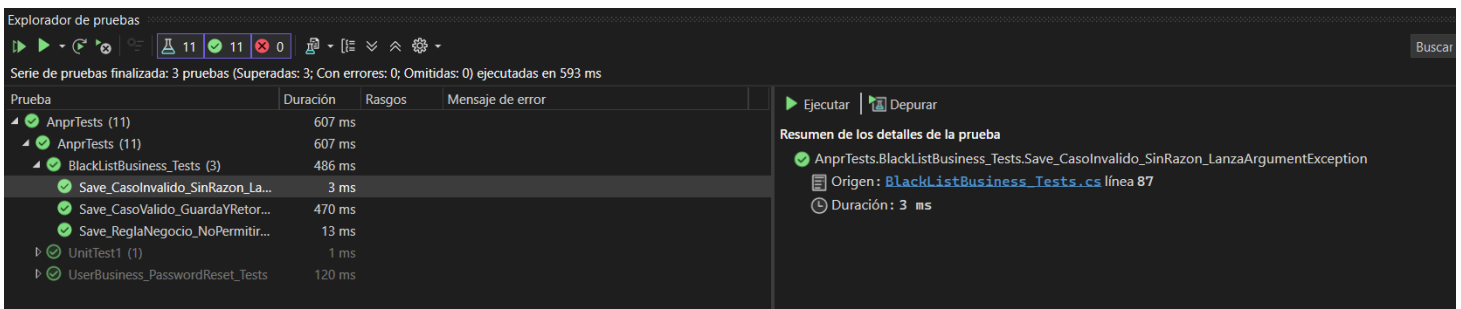
1. `Reason` = "" (vacío).

2. Ejecutar `Save(dto)` .

3. **Assert:** `ArgumentException` con mensaje relativo a “razón obligatoria”; **no** se invoca `Save` .

Resultado esperado: ✓ Prueba exitosa.

Evidencia:



CT-03 — Regla: no permitir duplicado por vehículo

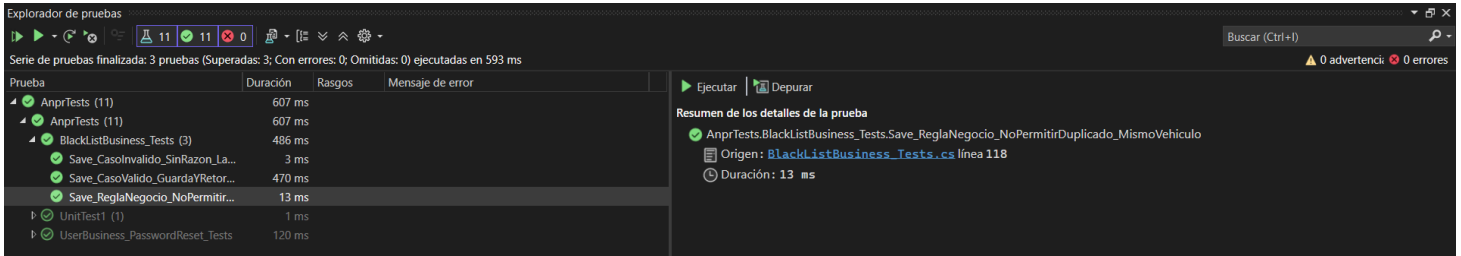
Precondición: `VehicleId` = 200 existe; `ExistsAsync(...)` → true .

Pasos:

1. Reason = "Reportes previos de incidente" .
2. Ejecutar Save(dto) .
3. **Assert:** InvalidOperationException con mensaje "ya está en la lista negra"; **no** se invoca Save .

Resultado esperado: ✓ Prueba exitosa.

Evidencia:



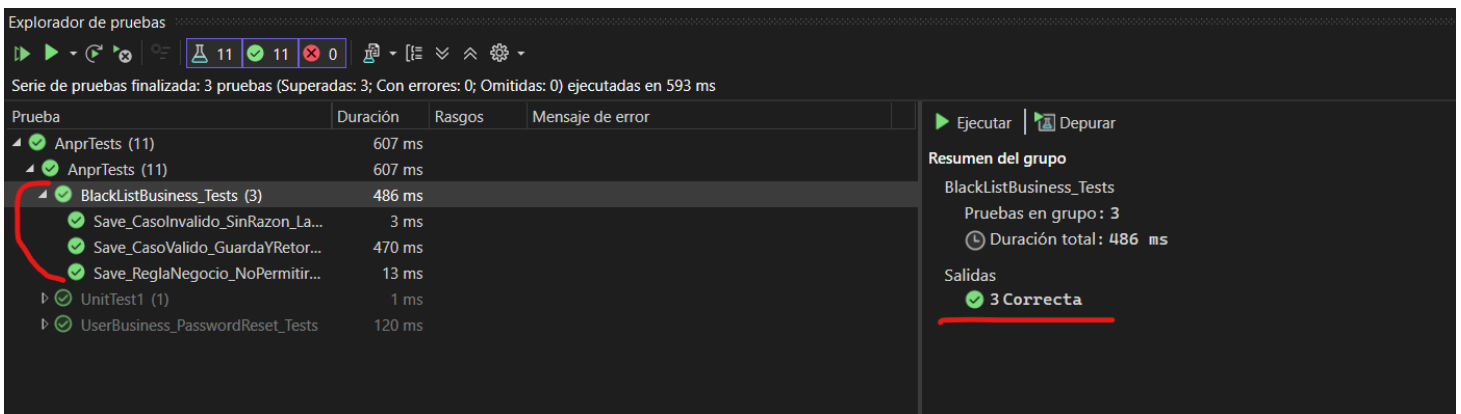
6. Resultado de ejecución

Runner: Test Explorer (Visual Studio).

Resultado: 3/3 pruebas superadas (0 fallidas).

Tiempo total: 486 ms

Capturas:



7. Criterios de aceptación

- Se exige **razón obligatoria** y se valida longitud máxima (250).
- Se bloquea el **duplicado** por VehicleId .
- En caso válido, se fija RestrictionDate y se retorna DTO con Id asignado.